

# Sujet d'examen UE NFE204 - Session 1

Année universitaire 2017-2018

Examen première session : 6 février 2018

## Consignes

Durée de l'examen : 3h00 ; Documents non autorisés ; Calculatrice autorisée

Les exercices sont indépendants les uns des autres. Merci de soigner votre écriture.

Ce sujet contient 3 pages.

*Important* : nous n'évaluons pas les réponses par la syntaxe. Ne vous inquiétez pas d'utiliser un point-virgule à la place d'un point d'exclamation ou autres détails mineurs

*L'ensemble de l'examen doit vous permettre de montrer que vous avez assimilé les principales notions vues en cours et en TP et que vous savez les mettre en application et les expliquer. La clarté, la concision, la précision des réponses sera donc appréciée.*

## Première partie : modélisation NoSQL (7 pts)

Pour cette partie, nous allons nous pencher sur le petit tutoriel proposé par la documentation en ligne de Cassandra, et consacré à la modélisation des données dans un contexte BigData. L'application (très simplifiée) est un service de musique en ligne, avec le modèle de données de la figure 1.

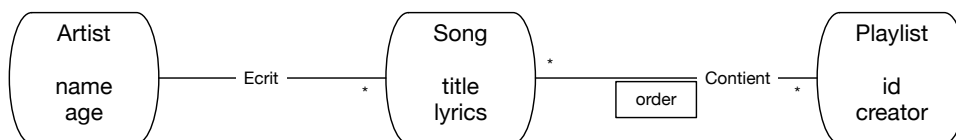


FIGURE 1 – Modélisation d'un service de musique en ligne

On a donc des chansons, chacune écrite par un artiste, et des *playlists*, qui consistent en une liste ordonnée de chansons.

1. Commencer par proposer le schéma relationnel correspondant à ce modèle. Il est sans doute nécessaire d'ajouter des identifiants. Donnez les commandes SQL de création des tables. (1 pt).
2. Le tutoriel Cassandra nous explique qu'il faut concevoir le schéma Cassandra en fonction des *Access patterns*, autrement dit des requêtes que l'on s'attend à devoir effectuer. Voici les trois *access patterns* envisagés :
  - (a) *Find all song titles*
  - (b) *Find all songs titles of a particular playlist*

Lesquels de ces *access patterns* posent potentiellement problème avec un système relationnel dans un contexte *BigData* et pourquoi ? Vous pouvez donner les requêtes SQL correspondantes pour clarifier votre réponse (1 pt).

3. Le tutoriel Cassandra nous propose alors de créer une unique table Cassandra suivante

```
CREATE TABLE playlists (
  id_playlist int,
  creator text,
  song_order int,
  song_id int,
  title text,
  lyrics text,
  name text,
  age int,
  PRIMARY KEY (id_playlist, song_order ) );
```

Discutez des avantages et inconvénients en répondant aux questions suivantes : combien faut-il d'insertions (au pire) pour ajouter une chanson à une *playlist*, en relationnel et dans Cassandra ? Que peut-on dire des requêtes qui affichent une *playlist*, respectivement en relationnel et dans Cassandra (donnez la requête si nécessaire) ? Combien de lignes dois-je mettre à jour quand l'âge d'un artiste change, en relationnel et en Cassandra ? Conclusion ? (2 pts)

4. Vous remarquez que l'identifiant de la table est composite (*compound* en anglais) : (*id\_playlist*, *song\_order* ). Voici ce que nous dit le tutoriel : *A compound primary key consists of the partition key and the clustering key. The partition key determines which node stores the data. Rows for a partition key are stored in order based on the clustering key.* Sur la base de cette explication, quelles affirmations sont vraies parmi les suivantes :

- (a) Une chanson est stockée sur un seul serveur (vrai/faux) ?
- (b) Les chansons d'une même *playlist* sont toutes sur un seul serveur (vrai/faux) ?
- (c) Les chansons stockées sur un serveur sont triées sur leur identifiant (vrai/faux) ?
- (d) Les chansons d'une même *playlist* sont stockées les unes après les autres (vrai/faux) ?

Faites un petit dessin illustrant les caractéristiques du stockage des *playlists* dans un système Cassandra distribué (2 pts).

5. Pour finir, reprenez les *access patterns* donnés initialement. Lesquels vont pouvoir être évalués très efficacement avec cette organisation des données, lesquels posent des problèmes potentiels de cohérence (1 pt) ?

## Deuxième partie : recherche d'information (5 pts)

On veut maintenant équiper notre système d'une fonction de recherche plein texte.

1. Un premier essai avec le langage CQL de Cassandra est évalué à partir d'un jeu de tests. On obtient les indicateurs du tableau suivant :

|         | Pertinent | Non pertinent |
|---------|-----------|---------------|
| Positif | 200       | 50            |
| Négatif | 100       | 1800          |

Sur 250 chansons ramenées dans le résultat, 200 sont pertinentes, 50 ne le sont pas, et il en manque en revanche 100 qui seraient pertinentes.

Quel est le rappel de votre système ? Quelle est sa précision ? (1 pt)

2. Essayons de faire mieux en associant Cassandra à Elasticsearch pour pouvoir faire de la recherche avec classement. Voici quelques extraits de grandes chansons françaises.

- (a) *Y a vraiment qu'l'amour qui vaille la peine*

- (b) *Que je t'aime, que je t'aime, que je t'aime*
- (c) *Dix ans de chaînes sans voir le jour, c'était ma peine forçat de l'amour*
- (d) *C'est (c'est) pas la peine c'est (c'est) (c'est) pas la peine*

On va considérer que la phase d'indexation unifie les termes “amour” et “aime”. Sans faire de calcul, expliquez le résultat attendu (classement compris) pour les recherches suivantes (2 pts)

- amour
- peine
- peine et aime

3. Un de vos collègues vous affirme que la similarité cosinus entre un vecteur-document  $v$  et un vecteur-requête  $q$  est définie par la formule suivante :

$$\cos\theta = \sum_{i=1}^n v[i] \times q[i]$$

Où  $x[i]$  désigne le nombre d'occurrences du  $i^{\text{ème}}$  terme dans un vecteur  $x$ . Expliquez pourquoi votre collègue a tort, et prouvez en prenant un des exemples de recherche ci-dessus que le résultat avec cette formule serait différent de celui attendu (2 pts).

### Troisième partie : Comprendre MapReduce tu devras (5 pts)

À une époque très lointaine, en pleine guerre intergalactique, deux Jedis isolés ne peuvent communiquer que par des messages cryptés. Le protocole de cryptage est le suivant : le **vrai** message est mélangé à  $N$  **faux** messages,  $N$  pouvant être très grand. Les messages sont tous découpés en mots, et l'ensemble est transmis en vrac sous la forme suivante :

```
{"idMessage": "Xh9788&&", "mot": "force"}
```

Les Jedi disposent d'une fonction secrète  $f()$  qui prend l'identifiant d'un message et renvoie **vrai** ou **faux**.

1. Vous devez fournir à Maître Y. le programme MapReduce qui reconstituera le contenu des **vrais** messages envoyés par Obiwan K. à partir d'un flux massifs de documents ayant la forme précédente. On accepte pour l'instant que les mots d'un message ne soient pas dans l'ordre. Donnez ce programme sous la forme que vous voulez, pourvu que ce soit clair (1 pt).
2. Quel est à votre avis (expliquez) le degré maximal de parallélisme que l'on peut obtenir pour ce traitement (2 pts) ?
3. Notez que, même après reconstitution, les mots d'un message sont dans n'importe quel ordre. Maître Y. sait lire et écrire des messages dans n'importe quel ordre, mais comment remettre dans l'ordre les mots des messages reçus par Obiwan K. ? Proposez une extension de la forme des messages et du programme MapReduce pour que chaque message soient reconstitué dans l'ordre (2 pts).

### Quatrième partie : questions de cours (3 pts)

1. Dans quelle architecture distribuée peut-on aboutir à des écritures conflictuelles ? Donnez un exemple.
2. Que signifie, pour une structure de partitionnement, être *dynamique*. Avons-nous étudié un système où le partitionnement n'était pas dynamique ?
3. Quel est le principe de la reprise sur panne dans Spark ?